

RM培训讲义

学习Cmake之前，你需要知道

1. VSCode的使用
2. 简单的编译原理
3. c++的多文件结构，函数、类的声明与定义

一些Ubuntu的设置

- 中文设置
- 虚拟机 安装VM-tools

```
bash
sudo apt install open-vm-tools-desktop
```

- 双系统 时间同步设置

```
sudo apt install ntpdate
sudo ntpdate time.windows.com
sudo hwclock --localtime --systohc
#将系统时间 (System Clock) 同步到硬件时钟 (Hardware Clock / RTC) , 并且指示硬件时钟
使用本地时间 (Local Time)
```

VSCode的安装使用

下载

<https://code.visualstudio.com/download>

下载 .deb 文件。

使用软件管理程序安装

设置中文

1. **打开命令面板**：使用快捷键 `Ctrl+Shift+P` 打开命令面板。
2. **配置显示语言**：在命令面板的输入框中输入 `Configure Display Language` (配置显示语言)，然后按回车键。

3. 选择语言:

- 此时会弹出一个 `locale.json` 文件, 其中会列出可用的语言选项。
- 选择简体中文, 按照好后按照提示重启VSCode

安装插件

需要安装C/C++,cmake,python插件

安装编译器

```
sudo apt install g++ cmake
```

简单了解编译原理

这里只是作简单的理论讲解, 只需要简单的了解即可。有能力的可以自学编译原理

为什么要学习编译原理:

我们写完的C++代码, 并不能直接运行。无论你用的时哪个IDE, 在运行前要进行一场“编译”。比如说, Dev-C++编写完程序, 需要“编译”或者“编译并运行”。今天学习的Cmake, 只是帮助我们构建和编译的工具。简单的了解编译原理, 才能明白为什么要用Cmake, Cmake的角色是什么。

计算机程序设计语言

相信你已经知道, 计算机的底层是二进制运算。所以对于计算机来说, 它们的语言是二进制的。相反, 我们人类的自然语言和二进制并不相通。所以需要有一个语言翻译的过程。这个翻译的过程就是**编译**。

要了解如何翻译, 首先要先了解这几种语言。

计算机程序设计语言分为计算机程序设计语言分为**机器语言 汇编语言 高级语言**

机器语言 可以被计算机直接理解的语言(是由01组成的序列) ;

比如 `C706 0000 0002` 将数值 2 存放到底址 `0000`

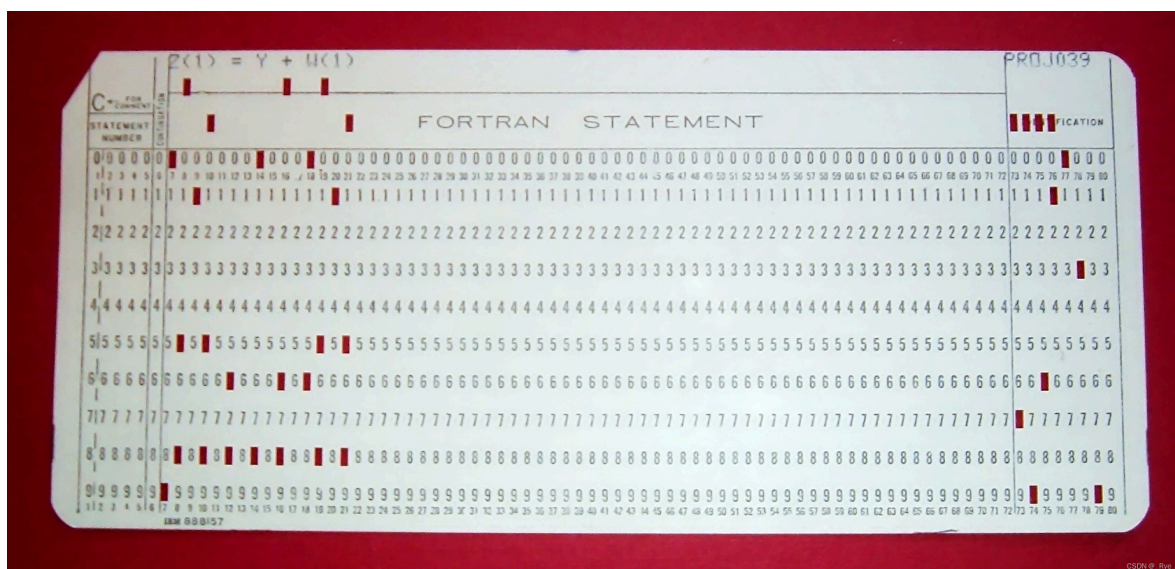
难记难写难读

汇编语言

引入了助记符
依赖于特定机器
编写效率低

高级语言 类似数学定义或者自然语言的简介形式 `x=2`

机器语言



早期的计算机只能使用**机器语言**，只能用二进制代码来编写程序。人们在打孔卡（Punched Card）的特定的位置上打洞或者不打洞，来代表“0”或者“1”。这样，要写的程序、要处理的数据，就变成一条条纸带或者一张张卡片，之后再交给当时的计算机去处理。

为什么早期的计算机程序要使用打孔卡，而不能像我们现在一样，用 C 或者 Python 这样的高级语言来写呢？因为计算机或者说 CPU 本身，**并没有能力理解这些高级语言**。即使在今天，我们使用的现代个人计算机，仍然只能处理所谓的“机器码”，也就是一连串的“0”和“1”这样的数字。

从硬件的角度来看，CPU 就是一个超大规模集成电路，通过电路实现了加法、乘法乃至各种各样的处理逻辑。如果我们从软件工程师的角度来讲，CPU 就是一个执行各种计算机指令

（Instruction Code）的逻辑机器。这里的计算机指令，就好比一门 CPU 能够听得懂的语言，我们也可以把它叫作**机器语言（Machine Language）**。

不同的 CPU 能够听懂的语言不太一样。比如，我们的个人电脑用的是 x86架构 的 CPU，手机用的是 ARM 架构的 CPU。这两者能听懂的语言就不太一样。类似这样两种 CPU 各自支持的语言，就是两组不同的**计算机指令集**。为了方便**人类**阅读和编写，计算机指令集**通常会用16进制来表示，但是实际上CPU处理的指令，仍然是二进制**。

`C7 45 FC 01 00 00 00`

字节	值	含义
1	C7	操作码: <code>mov r/m32, imm32</code> (将 32 位立即数写入 32 位内存/寄存器)
2	45	ModR/M 字节: 表示 <code>[ebp + r/m]</code> 形式的寻址, <code>-4</code> 的补码是 <code>FC</code>
3	FC	SIB 或位移: 这里是 <code>-4</code> 的 8 位补码表示
4-7	01 00 00 00	立即数 <code>1</code> 的小端序 (Little-Endian) 表示

如果我们在自己电脑上写一个程序, 然后把这个程序复制到自己的手机上, 肯定是没办法正常运行的, 因为这两者语言不通。而一台电脑上的程序, 简单复制一下到另外一台电脑上, 通常就能正常运行, 因为这两台 CPU 有着相同的指令集, 也就是说, 它们的语言相通的。

因此, **机器语言**就有几个问题: 一是单纯的 `C7 45 FC 01 00 00 00` 这样的指令, 你是很难看懂, 也很难记忆的。二是不同机器/CPU的指令集(语言)是不通的。除非经过专门的训练, 否则是没法操作计算机的。

汇编语言

为了方便使用, 于是人们发明了**汇编语言**, 它被认为是第二代计算机语言。它使用了ADD、MOV、SUB、DIV这些很容易记住英文表示的指令; 以及可以用符号表示内存地址。

前面说过, 计算机只能识别二进制的**机器码**, 所以需要将汇编语言**翻译**成机器语言。这个翻译的过程, 是由**汇编程序完成的**

但是汇编代码其实就是“给程序员看的机器码”, 仍然是面向机器的语言, 也正因为这样, 机器码和汇编代码是一一对应的。因此, 不同架构计算机的语言还是不通的, 程序员仍然要经过专门的训练才能使用计算机。

分配一个栈空间(例如, 相对于 `ebp` 的某个偏移量, 比如 `-4` 字节)。汇编代码大致如下:

```
mov DWORD PTR [rbp-0x4], 1
```

解释:

- `DWORD PTR`: 表示操作的是一个 32 位 (4 字节) 的双字。
- `[rbp-0x4]`: 表示变量 `a` 存储在栈帧中, 位于基址指针 `rbp` 向下 4 字节的位置。
- `1`: 立即数 1, 赋值给 `a`。

高级语言

编写高级语言不需要程序员了解实际的计算机硬件和底层指令集。只需要了解高级语言的语法和语义。 `int a = 1;`

程序员不需要直接对内存进行操作，使用变量这些简单易读的符号操作。

当然，计算机本身不能直接识别高级语言。因此，也需要将高级语言进行翻译。

高级语言

高级语言（High-level programming language）是高度封装的计算机语言，其语法设计更贴近人类的自然语言和数学逻辑。使用高级语言编程时，开发者无需深入了解底层硬件（如CPU指令集、内存地址等）的具体细节，而是可以专注于解决问题本身。

- **高可读性与易维护性：** 代码采用易于理解的词汇和符号来表达逻辑。

例如，`int a = 1;` 这行代码直观地定义了一个名为 `a` 的整数变量并赋值为1。这种特性使得程序更易于编写、阅读和后期维护。而不需要开发者手动地去分配内存地址。

- **简化内存管理：** 程序员通过变量等符号来操作数据，而无需直接管理具体的内存地址，大大降低了编程的复杂性。
- **良好的可移植性：** 高级语言编写的程序通常不依赖于特定的计算机硬件架构，只需稍作修改甚至无需修改，就可以在不同的计算机上运行。

计算机的CPU无法直接理解高级语言的语法。它只能识别由0和1组成的机器语言。因此，高级语言程序必须经过“翻译”才能执行。

一般来说，会将高级语言翻译成汇编语言程序（或者其他中间语言程序），然后再翻译成机器语言，由计算机执行。

操作系统与计算机语言的关系

计算机语言编写的程序并不是直接在“裸露”的硬件上运行，而是运行在**操作系统（Operating System, OS）**这个强大的软件平台之上。操作系统是硬件和所有应用程序之间的桥梁，它负责管理计算机的所有资源（如CPU时间、内存、文件、网络等）。

这种关系体现在以下几个关键点：

- **操作系统提供服务接口（API/系统调用）：**

程序执行过程中需要进行的很多操作，比如读取硬盘上的文件、在屏幕上显示文字、或者通过网卡发送数据，都不能由程序直接控制硬件来完成。这样做既不安全也不高效。相反，操作系统提供了一套标准的服务接口，称为**系统调用（System Calls）**。

- **语言作为“中间人”：**

无论是高级语言还是汇编语言，当它们需要执行上述底层操作时，最终都必须通过调用操作系统的接口来完成。例如，当你在C语言中使用 `fopen()` 函数打开一个文件时，C语言的标准库会把这个高级指令“翻译”成对应操作系统的特定系统调用（在Linux上可能是 `open`，在Windows上可能是调用 `CreateFile API`）。

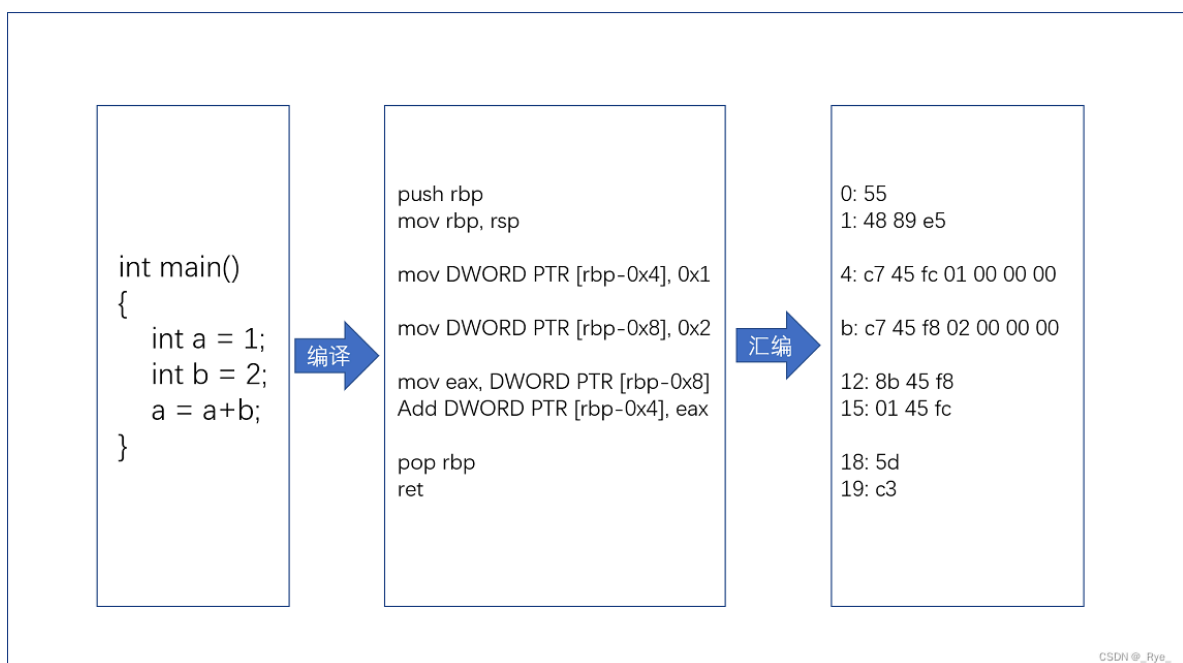
- **编译过程的“操作系统感知”：**

正因为不同操作系统（如Windows和Linux）提供的系统调用接口是完全不同的，所以在将高级语言编译成机器码时，编译器必须明确知道目标操作系统是什么。它会把同样的一行高级代码，根据目标平台的规则，翻译成调用相应系统服务的、完全不同的机器指令序列。这也是为什么为Windows编译的可执行文件（.exe）无法直接在Linux上运行的核心原因。

总而言之，计算机语言负责描述“做什么”，而操作系统则提供了“怎么做”的底层能力。语言通过调用操作系统的接口，将程序员的意图最终转化为硬件上的实际操作。

示例

项目	内容
C++ 代码	<code>int a = 1;</code> （局部变量）
汇编语言	<code>mov DWORD PTR [rbp-0x4], 1</code>
机器语言	<code>C7 45 FC 01 00 00 00</code> （十六进制）
说明	将 32 位立即数 <code>1</code> 存入栈中 <code>rbp-0x4</code> 处，代表变量 <code>a</code>



编译和解释

翻译高级语言的方式有两种：**编译**和**解释**

这两种方式的核心区别在于**代码翻译成机器语言的时机不同**。编译和解释就是两种最主要的翻译方式。

- **编译：**由**编译器**一次性将全部源代码转换成机器语言，生成一个独立的可执行文件（例如 Windows 下的 .exe 文件）。之后运行时，直接执行这个文件即可，运行效率较高。
- **解释：**由**解释器**逐行读取源代码，并即时翻译成机器指令来执行。这种方式灵活性更高，但通常运行速度比编译型语言慢。

编译型语言 (Compiled Languages)

核心思想：一次性翻译，多次运行。

编译型语言在程序执行**之前**，需要一个专门的“编译器”（Compiler）程序，将我们写好的所有源代码（例如 .c 文件）完整地转换成计算机可以直接执行的机器码，并打包成一个可执行文件（例如 Windows 下的 .exe 文件）。

- **过程：**
 1. **编写：**程序员编写源代码。
 2. **编译：**编译器将所有源代码一次性地、完整地翻译成目标机器码。
 3. **运行：**用户直接运行这个已经包含了机器码的可执行文件。程序运行时不再需要编译器或源代码。
- **优点：**
 - **运行速度快：**程序在运行时已经是CPU可以理解的机器码了，没有额外的翻译开销，所以执行效率非常高。
 - **脱离开发环境：**编译生成的可执行文件可以独立运行，不需要提供源代码和编译器。
- **缺点：**
 - **开发效率低：**每次修改代码后，都需要重新编译整个程序才能看到结果，对于大型项目来说，编译过程可能很耗时。
 - **跨平台性差：**一个为特定平台（如 Windows）编译好的程序，通常无法在其他平台（如 macOS 或 Linux）上运行，需要为每个平台单独编译。
- **典型例子：**C、C++、Go、Rust

打个比方：你把一本中文小说（源代码）完整地翻译成一本英文小说（可执行文件）。之后，任何讲英语的读者（CPU）拿到这本英文小说就能直接阅读，速度很快。但如果你发现中文原著里

有个错别字，你就必须把整本书重新翻译一遍。

解释型语言 (Interpreted Languages)

核心思想：一边翻译，一边运行。

解释型语言在程序**运行时**，需要一个“解释器”（Interpreter）程序，它会逐行读取源代码，然后即时地将每一行代码翻译成机器码并立即执行。

- **过程：**

1. **编写：**程序员编写源代码。
2. **运行：**用户通过解释器来运行源代码文件。解释器会一行一行地读取代码，翻译一行，执行一行，直到程序结束。

- **优点：**

- **开发效率高：**修改代码后可以立即运行，无需等待漫长的编译过程，调试和验证非常方便。
- **跨平台性好：**只要一个平台（如 Windows, macOS, Linux）上安装了对应的解释器，同一份源代码就可以直接在该平台上运行，无需任何修改。

- **缺点：**

- **运行速度慢：**每次执行程序时，都需要进行实时的翻译工作，这会带来额外的性能开销。
- **依赖解释器：**运行程序时，必须在目标环境中安装相应的解释器，并且需要提供源代码。

- **典型例子：**Python、JavaScript、Ruby、PHP

打个比方：你有一篇中文演讲稿（源代码），你需要一个同声传译（解释器）。当你开始演讲时，你说一句中文，同声传译就立刻翻译一句英文给听众（CPU）听。这个过程非常灵活，你可以随时修改你的演讲稿。但缺点是，每次演讲（运行）都需要这位翻译在场，而且整体速度会比直接读一篇翻译好的稿子要慢。

混合型语言（例如 Java）

现代很多语言采用了两者结合的方式来取长补短。以 **Java** 为例：

1. **编译：**Java 代码首先被 Java 编译器编译成一种称为“字节码”（Bytecode）的中间语言（`.class` 文件），而不是特定平台的机器码。
2. **解释/即时编译：**在运行时，Java 虚拟机（JVM）会充当解释器的角色，将字节码解释给不同平台的CPU听。为了提高效率，JVM 还会使用 **JIT（Just-In-Time，即时编译）** 技术，将

频繁执行的热点代码在运行时直接编译成原生机器码，从而获得接近编译型语言的执行速度。

这种方式既保留了高级语言的跨平台性（“一次编译，到处运行”），又通过 JIT 技术优化了运行性能。

g++编译器

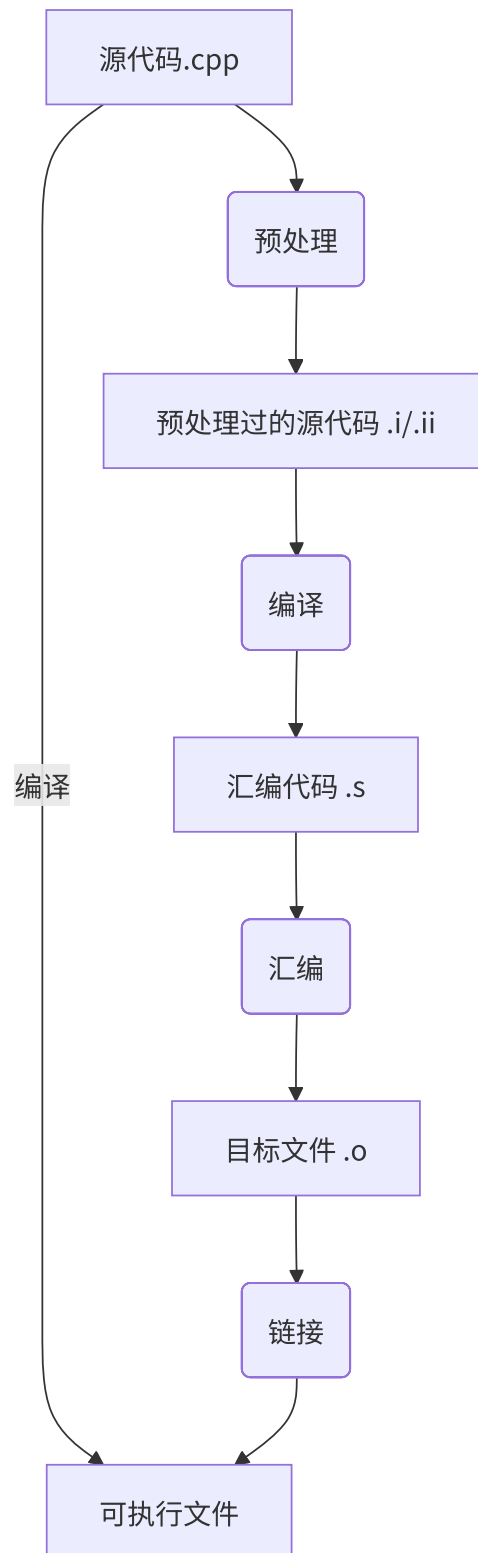
计算机的CPU只能直接理解由“0”和“1”组成的机器码，因此必须把我们编写的高级语言代码“翻译”成 **二进制文件** 才能执行。

g++编译器，就是把c++代码翻译成可执行文件（机器码）的工具

可执行文件就是二进制文件

在windows中，可执行文件的扩展就是 `.exe`

c的编译器是gcc，c++的编译器是g++



```
#预处理
g++ -E main.cpp -o main.i
#编译
g++ -S main.i -o main.s
#汇编
```

```
g++ -c main.s -o main.o
#链接
g++ main.o -o main
```

预处理

整合所有代码：将所以头文件，源代码整合成一个文件

编译

将源代码翻译成汇编代码

汇编

将汇编代码翻译成 机器语言

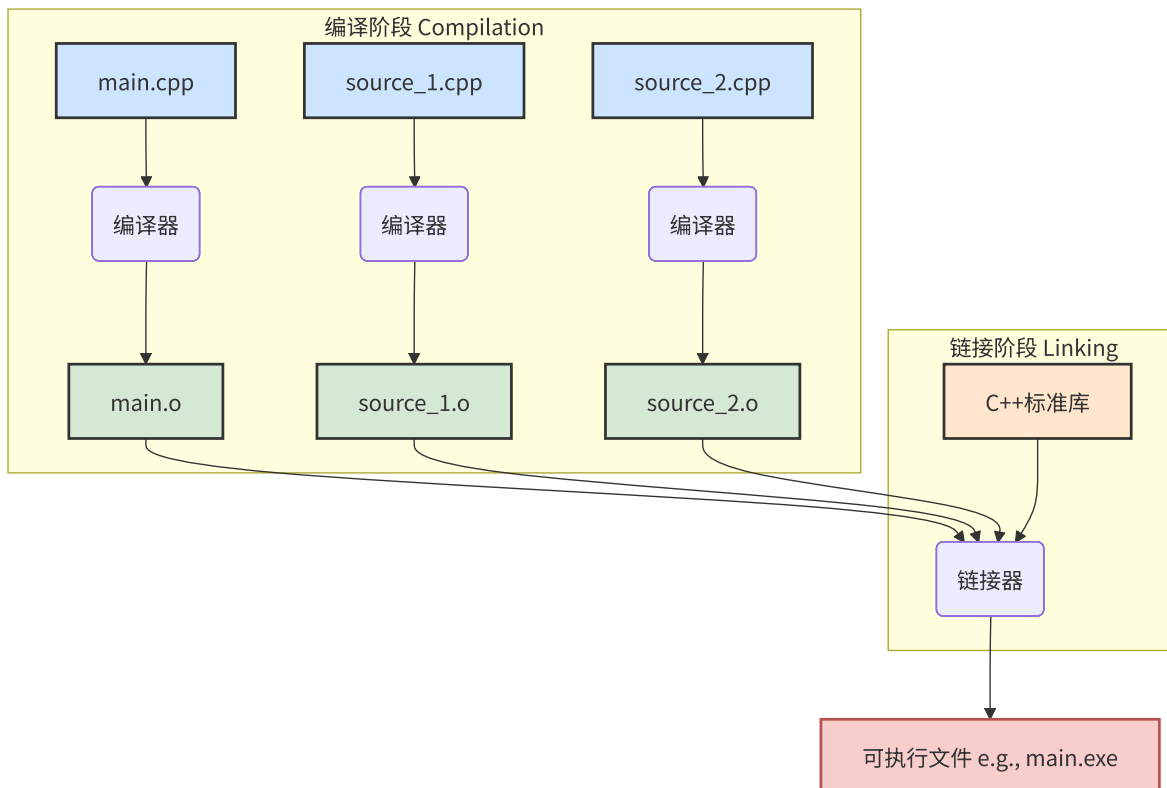
链接

具体看多文件的部分

多文件编译链接

编译流程

下面的图清晰地展示了从源代码到最终可执行文件的完整流程：



1. 编译 Compilation:

- 编译器如g++会**独立地**处理每一个 `.cpp` 源文件。
- `main.cpp` 被编译成一个名为 `main.o` 的**目标文件 Object File**。
- `source_1.cpp` 被编译成 `source_1.o`。
- `source_2.cpp` 被编译成 `source_2.o`。
- 每个目标文件 `.o` 都包含了它对应源文件的机器代码，但此时它们之间是相互独立的。例如，`main.o` 中对 `source_1` 和 `source_2` 函数的调用只是一个符号引用，并不知道这些函数的具体地址。

2. 链接 Linking:

- 再由链接器“组装”所有部件。
- 它将所有的目标文件 `main.o` , `calculator.o` , `logger.o` 作为输入。
- 同时，它也会链接程序所依赖的**C++标准库**例如，用于处理 `std::cout` 等功能的代码。
- 链接器进行**符号解析**找到 `main.o` 中调用的函数在 `calculator.o` 和 `logger.o` 中的具体实现和**重定位**为所有函数和变量分配最终的内存地址。
- 最终，所有部分被合并成一个单一的、完整的**可执行文件**例如 `my_app.exe` 或 `a.out`，这个文件就可以被操作系统加载并运行了。

库文件 `.so` `.a`

库可以简单理解为别人打包好的代码文件。

我们可以直接在代码中直接引用这些第三方库，不用我们重头开始造轮子。

特性	.a (静态链接库)	.so (动态链接库)
链接时间	编译时	运行时
文件大小	较大	较小
内存占用	较高	较低
更新方式	需重新编译应用程序	只需更新库文件
运行速度	启动速度快	启动速度稍慢
依赖性	无外部依赖	依赖于外部库文件
兼容性	兼容性问题较少	可能存在库版本不兼容问题

C++多文件工程

C++多文件项目的文件结构示例

这是一个常见的C++项目文件结构，通常包含头文件 `.h` 或 `.hpp`、源文件 `.cpp` 和一个主程序文件。

```
/ my_project
├── include/
│   ├── header_1.h
│   └── header_2.h
├── src/
│   ├── soucer_1.cpp
│   ├── source_2.cpp
│   └── main.cpp
└── Makefile (or build script)
```

- `main.cpp` : 程序的主入口，包含了 `main` 函数。
- `soucer_1.h` / `soucer_1.cpp` : `soucer_1`功能的声明和实现。

- `soucer_2.h` / `soucer_2.cpp` : `soucer_2`功能的声明和实现。

自定义头文件和源文件

头文件 “.hpp” 只需要包含函数的名称，函数调用的参数，不包含具体的函数实现

```
int add(int a,int b);
```

源文件：实现/定义 函数具体的细节

```
int tools::add(int a,int b) {  
    return a+b;  
}
```

在源文件中，通过 `#include "tools.hpp"` 引用头文件中的定义。

包含保护

```
#ifndef TOOLS_HPP  
#define TOOLS_HPP  
  
...  
  
#endif //TOOLS_HPP
```

- 在 C++ 中，.hpp 头文件使用 `#ifndef`、`#define` 和 `#endif` 这组预处理指令，是为了防止头文件被重复包含。确保无论一个头文件被 `#include` 多少次，其内容最终只会被编译器处理一次。
- `#ifndef TOOLS_HPP`：这是“if not defined”的缩写。预处理器会检查 `TOOLS_HPP` 这个宏是否未被定义。
- `#define TOOLS_HPP`：如果 `TOOLS_HPP` 确实未被定义，那么预处理器就会执行接下来的代码，首先就是定义 `TOOLS_HPP` 这个宏。
- `hpp`头文件内容：接着，头文件的所有实际代码（如类声明、函数原型等）都会被包含进来。
- `#endif`：标志着 `#ifndef` 条件编译块的结束。
- 如果没有 `#ifndef` 保护，会导致编译错误。

具体原因是**重定义 (Redefinition)**。

举一个简单的例子：

1. 你有一个头文件 `tools.hpp`，里面定义了一个类 `Tool`。
2. 你的另一个文件 `A.cpp` 需要使用 `Tool`，所以它包含了 `#include "tools.hpp"`。
3. 你的再一个文件 `B.cpp` 也需要使用 `Tool`，所以它也包含了 `#include "tools.hpp"`。

如果 `main.cpp` 同时包含了 `A.cpp` 和 `B.cpp` 中定义的（通过头文件间接包含），或者直接包含了两个需要 `tools.hpp` 的头文件，就会发生以下情况：

- 预处理器在处理代码时，会把 `tools.hpp` 的全部内容原封不动地复制粘贴到每一个 `#include` 它的地方。
- 结果就是，在最终交给编译器的代码中，`Tool` 这个类被定义了**两次或多次**。

C++ 语言规定，在同一个编译单元里，任何一个类、函数或变量都只能被定义一次。因此，编译器看到多个相同的定义时，就会报错，告诉你“`Tool` 被重定义了”，导致编译失败。

示例

```
/ lesson
├── include/
│   └── tools.h
├── src/
│   ├── tools.cpp
│   └── main.cpp
└── Makefile (or build script)
```

```
//main.cpp

#include "tools.hpp"
#include <iostream>

int main() {
    std::cout<<add(1,2);
    return 0;
}
```

```
//tools.hpp
```



```
#ifndef TOOLS_HPP
#define TOOLS_HPP

int add(int a,int b);

#endif //TOOLS_HPP
```

```
//tool.cpp

#include "tools.hpp"
int add(int a,int b) {
    return a+b;
}
```

用命令行编译：

```
g++ src/main.cpp src/tools.cpp -I include/ -o main
```

但是如果是大型项目，其可执行文件会特别多，还要引用第三方库，其最终的编译命令会变得很长

```
g++ main.cpp -o main -I/usr/include/opencv4 -lopencv_stitching -lopencv_alphamat -lopencv_aruco -lopencv_barcode -lopencv_bgsegm -lopencv_bioinspired -lopencv_ccalib
-lopencv_dnn_objdetect -lopencv_dnn_superres -lopencv_dpm -lopencv_face -lopencv_freetype -lopencv_fuzzy -lopencv_hdf -lopencv_hfs -lopencv_img_hash -lopencv_intensity_transform
-lopencv_line_descriptor -lopencv_mcc -lopencv_quality -lopencv_rapid -lopencv_reg -lopencv_rgbd -lopencv_saliency -lopencv_shape -lopencv_stereo -lopencv_structured_light
-lopencv_phase_unwrapping -lopencv_superres -lopencv_optflow -lopencv_surface_matching -lopencv_tracking -lopencv_highgui -lopencv_datasets -lopencv_text -lopencv_plot -lopencv_ml
-lopencv_videostab -lopencv_videoio -lopencv_viz -lopencv_wechat_grocery -lopencv_ximgproc -lopencv_videon -lopencv_xobjdetect -lopencv_objdetect -lopencv_calib3d -lopencv_imgcodecs
-lopencv_features2d -lopencv_dnn -lopencv_flann -lopencv_xphoto -lopencv_photo -lopencv_imgproc -lopencv_coreiyy
```

Cmake

Cmake

CMake 属于**编译前端之外的、用于自动化和管理整个编译构建流程的工具**。

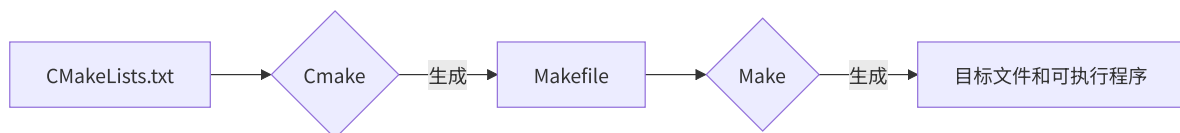
在编译原理之中，CMake 并不属于核心的编译阶段，而是处于编译过程之前的一个重要环节，即**构建过程（Build Process）**。

构建 (Build)：这是一个更宏观的概念，它不仅包含编译，还包括了将项目从源代码转换成可执行文件或库所需的全部步骤。对于一个复杂的项目，这可能涉及：

- 管理源文件之间的依赖关系。
- 管理项目对第三方库的依赖。
- 根据不同的操作系统或环境配置编译选项。
- 调用编译器来编译各个源文件。
- 调用链接器 (Linker) 将编译生成的目标文件和库链接成最终的可执行文件或库。

具体来说，CMake 是一个**跨平台的构建系统生成工具 (Build System Generator)**。
它的主要职责是：

1. **读取开发者编写的 CMakeLists.txt 配置文件**：开发者在这个文件中使用相对简单的语句来描述项目的源文件、依赖库、编译选项等信息。
2. **生成特定平台的构建文件**：CMake 会根据当前的环境（如 Windows、Linux、macOS）和选择的编译器（如 GCC、Clang、Visual Studio），自动生成相应平台和构建工具能够理解的文件，例如 Unix/Linux 下的 Makefile 文件或 Windows 下的 Visual Studio 项目文件 (.sln)。
3. **协调编译过程**：生成构建文件后，开发者就可以使用平台原生的构建工具（如 make 命令或在 Visual Studio 中点击“生成”）来执行实际的编译和链接工作。这些构建工具会根据 CMake 生成的规则去调用编译器（如 GCC）来完成编译任务。



Make

make 的核心是读取当前目录下的 Makefile 文件。

Makefile 文件定义了构建项目的规则。

CMakeList.txt

[!Warning]

CMakeLists.txt 的文件名大小写必须完全一致

CMakeLists.txt 文件是 CMake 的核心配置文件，其作用是作为项目的构建蓝图。它通过一系列指令来告诉 CMake 您的项目包含哪些源文件，需要生成什么目标（例如可执行文件或库），以及这些目标依赖于哪些其他的库和头文件。简而言之，它定义了项目的结构和构建规则，

CMake 会读取这个文件的内容来生成适用于特定编译器和平台的实际构建文件（如 Makefile 或 Visual Studio 项目）。

写法一：直接生成可执行文件

```
cmake_minimum_required(VERSION 3.22) #最低版本要求

project(lesson_cmake) #项目名字

include_directories(include) #定义头文件位置  cmake从CMakeList.txt所在目录下查找

add_executable(lesson_cmake src/main.cpp src/tools.cpp) #可执行文件
```

写法二：先生成库，再链接到可执行文件

```
cmake_minimum_required(VERSION 3.22) #最低版本要求

project(lesson_cmake) #项目名字

include_directories(include) #定义头文件位置  cmake从CMakeList.txt所在目录下查找

add_executable(main src/main.cpp) #可执行文件

add_library(tools src/tools.cpp)

target_link_libraries(main tools)
```

此方法会额外生成一个库文件 `libtools.a`（静态库）

第一种写法更直接，适用于小型、简单的项目。第二种写法更模块化，是更推荐的实践方式，尤其适用于代码规模变大、逻辑更复杂的项目。

编译

```
mkdir build
cmake -B build
make -C build/ -j2
./build/main
```

```
strace cmake -B build 2>&1 | grep -i open
```

往后修改了代码（cpp和hpp的内容）直接执行 `make -C build/ -j2` 即可重新编译

若修改了 `CMakeList.txt`，需要重新执行 `cmake -B build`

使用第三方库的写法

以openCV为例

```
cmake_minimum_required(VERSION 3.8)

# 设置项目名称
project(MyOpenCVProject)

# 查找OpenCV库
# REQUIRED关键字表示如果找不到OpenCV，CMake配置过程将失败
find_package(OpenCV REQUIRED)

## 手动将OpenCV的头文件目录添加到全局包含路径中（现在的cmake可以不用这么写，直接写
target_link_libraries即可）
# include_directories(${OpenCV_INCLUDE_DIRS})

# 添加你的可执行文件
# 假设你的源文件名为 main.cpp
add_executable(my_app main.cpp)

# 将你的可执行文件链接到OpenCV库
# ${OpenCV_LIBS} 是由find_package(OpenCV)命令自动创建的变量，包含了所有需要的库文件
target_link_libraries(my_app PRIVATE ${OpenCV_LIBS})
```

ROS2中

```
find_package(rclcpp REQUIRED)      #c++ ros2 库
ament_target_dependencies(test_c rclcpp std_msgs)
```